

Assignment #3

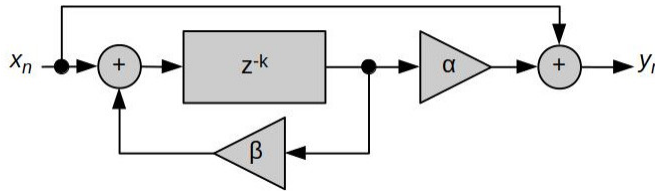
CS 246, FALL 2026

Due Friday, September 25

Simple echoes can be cast in the form of a digital filter. For a single tap echo, the transfer function would simply be

$$H(z) = 1 + \alpha z^{-k} \quad (1)$$

where α ($0 \leq \alpha \leq 1$) is the mixing factor of the echo, and k gives the index of the delay: for a delay of T seconds, we would take $k = \lfloor RT \rfloor$, where R is the sampling rate. A repeated echo can be obtained by feeding back the output into the input of the filter as in the following block diagram:



Note that this is different from the simple feedback delay previously discussed in class. The above block diagram leads to the transfer function

$$H(z) = \frac{1 + (\alpha - \beta)z^{-k}}{1 - \beta z^{-k}} \quad (2)$$

where β ($0 \leq \beta \leq 1$) is the feedback factor. Observe that if $\beta = 0$, then we recover the simple one tap echo filter in equation (1). And if $\alpha = \beta$, we get simple feedback delay.

For this part of the assignment you are to implement an echo filter with feedback. I will give you the file `Echo.h` that declares the class

```
class Echo : public Filter {
public:
    Echo(float Tmax=1, float R=44100.0f);
    void setDelay(float T);
    void setMix(float a);
    void setFeedback(float b);
    float operator()(float x);
private:
    int max_samples;
    RingBuffer xvalues, yvalues;
    float rate, mix, feedback, offset;
};
```

The `Filter.h` and `RingBuffer.h` header files have been included. The `Filter` interface class is described at the end of the handout.

`Echo(Tmax,R)` — (constructor) creates an echo filter object. The argument `Tmax` specifies the maximum delay value (in seconds), and the argument `R` is the sampling rate.

`setDelay(T)` — sets the delay of the filter (in seconds). The value of `T` is assumed to be between 0 and `Tmax` seconds, where `Tmax` is the value specified in the constructor (no error checking is performed).

`setMix(alpha)` — sets the mixing factor, the parameter α in equation (2).

`setFeedback(beta)` — sets the feedback factor, the parameter β in equation (2).

`operator(x)` — returns the output of the echo filter, given the input value `x`. Note that to compute the n -th output sample from the recurrence relation corresponding to (2):

$$y_n = x_n + (\alpha - \beta)x_{n-k} + \beta y_{n-k},$$

the values of $x_{n-1}, x_{n-2}, \dots, x_{n-k}$ as well as $y_{n-1}, y_{n-2}, \dots, y_{n-k}$ need to be stored. For this reason, you will need to maintain two circular buffers: one for the input values, and one for the output values.

For Full Credit

If you complete the above two tasks as stated, you will receive a *maximum* grade of 92%. To receive full credit (maximum of 100%), you will have to allow for a continuous range of delay times.

In equation (2), the parameter k is assumed to be an integer: it specifies a delay of k *samples*. Indeed, for a given delay time of T seconds, we take $k = \lfloor RT \rfloor$. In effect, this means that the delay time T is forced to be an integer multiple of the sampling period $1/R$. For many applications, this is sufficient. However, there are instances where we would like to allow k to have non-integral values; i.e., we take $k = RT$, without truncating to an integer. To implement this, you will have to use linear interpolation between adjacent samples in the circular buffers for both the input and output values.

Filter interface

For the echo filter and all of the filters that we will implement in the remaining homework assignments, we will derive from the following interface class contained in the file `Filter.h`.

```
class Filter {
public:
    virtual ~Filter(void) { }
    virtual float operator()(float x) = 0;
};
```

That is, any filter that we implement will be guaranteed to provide an overloaded `()` operator. The initial (0-th) call to this function will return the output $y = y_0$ of the filter at time

$t = 0$, given an input value of $x = x_0$; i.e., returns y_0 , with x_0 as input. The n -th call to this function will return the output of the filter $y = y_n$ at time $t_n = n/R$, given an input value of $x = x_n$. That is, it returns y_n , with x_n as input. Here R is the sampling rate.

What to turn in

For this assignment, you should turn in a single source file named `Echo.cpp`. You may include the header files `Filter.h`, `RingBuffer.h`, `Echo.h`, as well as any standard C++ header file. Note that I will be compiling your code with my implementation of the `RingBuffer` class from the previous assignment.